

Documentação Técnica Completa

Arquitetura, banco de dados, API e lições aprendidas

Gerado em: 22/09/2026

1. Arquitetura Geral

O sistema segue uma arquitetura monolítica simples e deliberada — adequada para uma equipe pequena e fácil de manter sem DevOps especializado.

Camada	Tecnologia	Descrição
Frontend	Vanilla JavaScript SPA	Single Page Application sem framework. Toda navegação é client-side via App.navigate().
Backend	Node.js 18+ / Express	API REST. Todas as rotas em /backend/routes/. Autenticação JWT em todas as rotas exceto login.
Banco (produção)	PostgreSQL via Supabase	Shared Pooler na porta 6543. Nunca usar conexão direta (porta 5432) — incompatível com Supabase.
Banco (local)	sql.js (SQLite em memória)	Apenas para desenvolvimento. Salvo em /data/compras.db. Não é usado em produção.
Hospedagem	Render (free tier)	Deploy automático via GitHub. Dorme após 15 min de inatividade — primeiro acesso demora.
Repositório	GitHub	github.com/bru-compressores/compras — branch main.

2. Variáveis de Ambiente

Configuradas no Render → serviço "compras" → Environment:

Variável	Obrigatória	Descrição
DATABASE_URL	Sim	Connection string PostgreSQL via Supabase Shared Pooler. Formato: postgresql://postgres.XXXX:SENHA@aws-1-sa-east-1.pooler.supabase.com:6543/postgres
JWT_SECRET	Sim	Chave secreta para assinar tokens JWT. Se alterada, todos os usuários são deslogados. Valor atual: BRUcompressores@2026#SecretKey!xyz789
PORT	Não	Porta do servidor. O Render define automaticamente. Padrão local: 3000.

3. Banco de Dados — Tabelas

Tabela: usuarios

Usuários do sistema

Coluna	Tipo / Descrição
id	SERIAL PK
nome	TEXT
email	TEXT UNIQUE
senha_hash	TEXT — bcrypt
papel	TEXT — 'admin' ou 'operador'
ativo	INTEGER — 1=ativo, 0=inativo
criado_em	TIMESTAMP

Tabela: ordens_servico

O.S. importadas do Primam

Coluna	Tipo / Descrição
id	SERIAL PK
numero_os	TEXT UNIQUE — número do Primam
cliente	TEXT
equipamento	TEXT — modelo + série + marca
data_abertura	TEXT — data da O.S. no Primam
data_conclusao_estimada	TEXT
status	TEXT — 'Aberta', 'Aguardando Aprovação Técnica', 'Aguardando peças', 'Peças separadas', 'Concluída', 'Ocultada'
prioridade	TEXT — 'Alta', 'Média', 'Baixa'
tipo	TEXT — 'OS' ou 'Pedido'
data_entrada_compras	TIMESTAMP — capturado ao importar PDF da O.S.
data_triagem_concluida	TIMESTAMP — capturado ao finalizar triagem
data_primeiro_pedido	TIMESTAMP — capturado ao 1ª peça ir para Pedido realizado
data_todas_pedidas	TIMESTAMP — capturado ao avançar para Aguardando peças
data_entrega_completa	TIMESTAMP — capturado ao avançar para Peças separadas

Tabela: pecas_os

Peças de cada O.S.

Coluna	Tipo / Descrição
id	SERIAL PK
os_id	INTEGER FK → ordens_servico (CASCADE DELETE)
codigo	TEXT — código do Primam
referencia	TEXT — referência do fornecedor (ex: KX500-007)
codigo_fabricante	TEXT — código do fabricante (ex: KA600-030TB)
descricao	TEXT NOT NULL

quantidade	NUMERIC(10,3) — aceita decimais (ex: 0.5 KG)
preco_unitario	NUMERIC — preço de venda (ERP)
preco_cotado	NUMERIC — melhor cotação obtida
preco_fechado	NUMERIC — valor final negociado com fornecedor
fornecedor_id	INTEGER FK → fornecedores
status_entrega	TEXT — 'Aguardando Triagem', 'Separado (Almoxarifado)', 'Pendente', 'Em cotação', 'Bloqueado no Fornecedor'
numero_pc	TEXT — ex: 'PC-1820'
data_compra	TEXT — data emissão do PC
data_entrega_prevista	TEXT — previsão de entrega
numero_rastreio	TEXT
transporte	TEXT — nome da transportadora
observacoes	TEXT
criado_em / atualizado_em	TIMESTAMP

Tabela: fornecedores

Cadastro de fornecedores

Coluna	Tipo / Descrição
id	SERIAL PK
nome	TEXT NOT NULL
cnj / contato / telefone / email / cidade / estado	TEXT
observacoes	TEXT

Tabela: historico_status

Auditoria de mudanças de status das O.S.

Coluna	Tipo / Descrição
id	SERIAL PK
os_id	INTEGER FK → ordens_servico (CASCADE DELETE)
status_anterior / status_novo	TEXT
observacao	TEXT — motivo da mudança
usuario_id	INTEGER FK → usuarios
criado_em	TIMESTAMP

Tabela: comentarios_peca

Comentários por peça

Coluna	Tipo / Descrição
id	SERIAL PK
peca_id	INTEGER FK → pecas_os (CASCADE DELETE)
usuario_id	INTEGER FK → usuarios
texto	TEXT NOT NULL

criado_em	TIMESTAMP
-----------	-----------

Tabela: configuracoes

Configurações do sistema (chave-valor)

Coluna	Tipo / Descrição
chave	TEXT PK — ex: markup_verde, markup_laranja
valor	TEXT
atualizado_em	TIMESTAMP

4. Rotas da API

Todas as rotas exceto /api/auth/login exigem header Authorization: Bearer {token}

Autenticação (/api/auth)

Rota	Descrição
POST /login	Login. Body: {email, senha}. Retorna {token, usuario}

Ordens de Serviço (/api/os)

Rota	Descrição
GET /	Lista O.S. com filtros: status, prioridade, tipo, busca, pagina, limite
GET /:id	Detalhe da O.S. com pecas, historico
POST /	Criar nova O.S.
PUT /:id	Atualizar O.S. — registra historico se status mudar
DELETE /:id	Excluir O.S. e peças (CASCADE)
POST /:id/duplicar	Duplica O.S. com peças sem preços

Peças (/api/pecas)

Rota	Descrição
GET /entregas-pendentes	Peças ativas para Rastreamento (exclui Entregue, Cancelado, Separado, Ag. Triagem)
GET /	Lista peças com filtros os_id, status_entrega
POST /	Adicionar peça — dispara verificação de avanço automático da O.S.
PUT /:id	Atualizar peça — dispara avanço automático + captura datas de lead time
DELETE /:id	Excluir peça

Importação (/api/importar-pdf e /api/importar-pc)

Rota	Descrição
POST /importar-pdf	Importa PDFs de O.S. do Primam. Suporta Layout 1 (código+descr. grudados) e Layout 2 (colunas separadas)
POST /importar-pc	Importa PDFs de Pedido de Compra. Vincula peças via código + requisição (opcional). Preenche preco_fornecedor

Triagem (/api/triagem)

Rota	Descrição
GET /	Lista O.S. com peças em "Aguardando Triagem"
GET /:os_id	Peças de uma O.S. para triagem
PUT /peca/:id	Decisão: {decisao: "separado" "comprar"}
POST /:os_id/finalizar	Finaliza triagem — captura data_triagem_concluida

Saving e Lead Times (/api/saving)

Rota	Descrição
GET /	Dados de saving: totais, por mês, por fornecedor, top peças
GET /lead-times	Lead times médios e histórico por O.S.

Dashboard (/api/dashboard)

Rota	Descrição
GET /dashboard	KPIs, alertas, O.S. recentes, peças atrasadas, O.S. vencendo, contadores de triagem/aprovação/bloqueio

Outros

Rota	Descrição
GET /api/fornecedores	Lista fornecedores com contagem de peças
POST /api/backup/exportar-json	Exporta todo o banco em JSON
POST /api/backup/restaurar	Restaura backup JSON (assíncrono)
GET /api/relatorios/economia	Relatório de economia por fornecedor e mês
GET /api/pdf-os/:id	Gera PDF da O.S. para impressão

5. Padrões de Código

5.1 Backend — Padrão obrigatório de queries

TODAS as queries devem usar `Promise.resolve()` para funcionar tanto com SQLite (síncrono) quanto com PostgreSQL (assíncrono):

```
const q = (db, sql, ...p) => Promise.resolve(db.prepare(sql).get(...p)); // retorna 1 linha
const qa = (db, sql, ...p) => Promise.resolve(db.prepare(sql).all(...p)); // retorna array
const qr = (db, sql, ...p) => Promise.resolve(db.prepare(sql).run(...p)); // insert/update/delete // Uso correto:
const os = await q(db, 'SELECT * FROM ordens_servico WHERE id = ?', id);
const pecas = await qa(db, 'SELECT * FROM pecas_os WHERE os_id = ?', os.id);
await qr(db, 'UPDATE pecas_os SET status_entrega=? WHERE id=?', novoStatus, pecaId);
```

5.2 Valores numéricos do PostgreSQL

PostgreSQL retorna `COUNT()` e valores numéricos como `STRING`, não como `number`. Sempre converter:

```
// Errado: const total = result.total; // pode ser "28" (string) // Correto:
const total = parseInt(result.total) || 0;
const valor = parseFloat(result.valor) || 0;
```

5.3 Frontend — Estrutura de uma página

Cada tela é um objeto JavaScript em `/frontend/assets/js/pages/NomePagina.js`:

```
const PageExemplo = {
  dados: null,
  async render(params) {
    // 1. Define botões do topbar
    document.getElementById('topbar-actions').innerHTML = '...';
    // 2. Mostra loading
    document.getElementById('content').innerHTML = 'Carregando...';
    // 3. Busca dados da API
    this.dados = await Api.get('/minha-rota');
    // 4. Renderiza HTML
    document.getElementById('content').innerHTML = this.renderHTML();
  },
  renderHTML() {
    return '...' + this.dados.map(d => ...).join('') + '';
  }
};
```

5.4 Sincronização obrigatória: colunas vs células

Na tabela de peças em `detalhe-os.js`, o array `_colsPecas` e as células da linha devem ter EXATAMENTE o mesmo número de entradas, na mesma ordem. Sempre verificar após mudanças:

```
Verificar no terminal: node -e com regex para contar { key: no array _colsPecas e contar td na linha da tabela.
Numero de colunas deve ser igual ao numero de celulas (exceto botao excluir).
```

6. Lições Aprendidas — Bugs Reais que Já Corrigimos

Esta seção documenta problemas reais encontrados durante o desenvolvimento. Vale muito a pena ler antes de mexer no sistema.

Bug: database.js — O arquivo que mais causou problemas

O arquivo database.js tem tendência a "regredir" para uma versão antiga síncrona (SQLite puro) que não funciona com PostgreSQL. Sempre que o sistema der erro "Use getAsync no PostgreSQL", verificar este arquivo primeiro.

Sintoma: Erro "Use getAsync no PostgreSQL" nos logs do Render.

Solução: Substituir database.js pela versão correta com async/await e Promise.resolve().

Bug: Vacuous truth do JavaScript — [].every() retorna true

JavaScript retorna true para [].every(qualquer_condição). Isso causou avanço automático indevido da O.S. quando TODAS as peças eram do almoxarifado (Separado) e nenhuma precisava de compra.

Sintoma: O.S. avançava para "Aguardando peças" sem ter peças para comprar.

Solução: Sempre verificar ativas.length > 0 antes de .every(). Código atual protege contra isso.

Bug: Supabase — IPv6 no Render free

A conexão direta do Supabase (porta 5432) usa IPv6, que o Render free não suporta. Causa erro "ENETUNREACH" nos logs.

Sintoma: Login falha com erro ENETUNREACH nos logs.

Solução: Usar sempre o Shared Pooler (porta 6543) na DATABASE_URL. Formato: postgres://postgres.PROJETO:SENHA@aws-1-sa-east-1.pooler.supabase.com:6543/postgres

Bug: PostgreSQL retorna números como string

Diferente do SQLite, o PostgreSQL retorna COUNT() e valores NUMERIC como strings JavaScript. Código que funciona local falha em produção.

Sintoma: NaN aparece em cálculos, ou erro ".toFixed is not a function".

Solução: Sempre usar parseInt() ou parseFloat() ao receber valores do banco.

Bug: const dentro de eval() não é acessível

O extrator de PDF usa eval() para reuso de código. Variáveis declaradas com const fora da função não ficam acessíveis dentro do eval(). Isso causou falha silenciosa na extração de peças.

Sintoma: 0 peças importadas mesmo o PDF sendo válido.

Solução: Mover todas as constantes (regex, ANOS, UNIDS) para DENTRO da função extrairOSPrimam().

Bug: Quantidades decimais — INTEGER vs NUMERIC

A coluna quantidade foi criada como INTEGER. PDFs com quantidades como 0,50 KG causavam erro "invalid input syntax for type integer".

Sintoma: Importação de PDF falha com erro de tipo.

Solução: ALTER TABLE pecas_os ALTER COLUMN quantidade TYPE NUMERIC(10,3);

Bug: Mistura de pastas no deploy

Ao extrair o ZIP do GitHub (compras-main.zip), a pasta interna se chama "compras-main". Copiar ela inteira para dentro do repositório git criou uma pasta "compras-main" duplicada dentro do projeto.

Sintoma: `git add .` inclui arquivos de `compras-main/*` além dos arquivos corretos.
Solução: Sempre usar `xcopy` para copiar APENAS o conteúdo para dentro de `controle-compras_1/compras/`

Bug: Layout 1 do PDF — código grudado com descrição

No Layout 1 do Primam, código e descrição aparecem na mesma linha sem separador ("7544CORREIA"). A regex original só tratava código isolado ou tudo na mesma linha com unidade.

Sintoma: 0 peças importadas em O.S. do Layout 1 após refatoração do extrator.
Solução: Adicionar `RE_COD_DESC = /^(\\d{3,6})([A-Z].+)$/` para capturar código grudado com descrição.

Bug: Colunas vs células desalinhadas na tabela

A tabela de peças tem 18 colunas definidas em `_colsPecas`. Em várias refatorações, o número de células na linha ficou diferente das colunas definidas, causando desalinhamento visual.

Sintoma: Dados aparecem em colunas erradas (fornecedor na coluna de PC, etc.).
Solução: Sempre contar e sincronizar manualmente. 18 colunas = 18 células + 1 botão excluir = 19 total.